Experiment 2: Email Spam/Ham Classification using Naïve Bayes and KNN

Machine Learning Algorithms Laboratory (ICS1512)
M.Tech (Integrated) Computer Science and Engineering, Semester V
Sri Sivasubramaniya Nadar College of Engineering, Chennai

1 Aim and Objective

To build and compare email spam/ham classifiers using **Naïve Bayes** (Gaussian, Multinomial, Bernoulli variants) and **K-Nearest Neighbors (KNN)**, and to:

- Compare Gaussian, Multinomial and Bernoulli Naïve Bayes.
- Study the effect of varying k in KNN.
- Compare KDTree and BallTree neighbor search structures.
- Optimize KNN using `GridSearchCV` and `RandomizedSearchCV`.
- Evaluate computational complexity and execution time, both theoretically and experimentally.
- Evaluate model stability using K-Fold Cross Validation.

2 Theory

2.1 Naïve Bayes

Naïve Bayes is a probabilistic classifier based on Bayes' theorem, assuming conditional independence of features given the class:

$$P(C \mid X) = \frac{P(X \mid C)\, P(C)}{P(X)} \tag{1}$$

where C is the class label and $X = (x_1, x_2, \ldots, x_d)$ is the feature vector. Three variants were used, differing in the assumed distribution of $P(x_i \mid C)$:

- **Gaussian NB** assumes each feature is normally distributed within a class.
- **Multinomial NB** models feature counts/frequencies, suited to word/character frequency data.
- **Bernoulli NB** models binary presence/absence of a feature.

2.2 K-Nearest Neighbors (KNN)

KNN is a lazy, instance-based learning algorithm. A query point is classified by majority vote among its k nearest neighbors in the training set, under a chosen distance metric (e.g. Euclidean, Manhattan). No explicit model is built at training time; all computation is deferred to prediction time.

2.3 Neighbor Search Structures: KDTree and BallTree

Brute-force KNN search is $O(nd)$ per query. **KDTree** and **BallTree** are spatial index structures that partition the training data to accelerate nearest neighbor queries to $O(\log n)$ on average, at the cost of an $O(n \log n)$ build time. KDTree partitions space along axis-aligned hyperplanes and tends to work best in low-to-moderate dimensions, while BallTree partitions data into nested hyperspheres and is often more robust as dimensionality grows.

2.4 Hyperparameter Search: GridSearchCV vs. RandomizedSearchCV

`GridSearchCV` exhaustively evaluates every combination in a specified parameter grid using cross-validation, guaranteeing the best combination within that grid at the cost of computation time that grows multiplicatively with the number of parameters and values. `RandomizedSearchCV` instead samples a fixed number of random combinations, trading an exhaustive guarantee for substantially lower computational cost, and scales better to large search spaces.

3 Dataset

The **Spambase** dataset from the **UCI Machine Learning Repository** (dataset id 94, https://archive.ics.uci.edu/dataset/94/spambase) was used.

Table 1: Dataset Summary

Item	Value
Total samples	4601
Features	57 (numeric: word/character frequencies, capital-run-length statistics)
Missing values	0
Duplicate rows	391 (retained)
Class balance	Ham (0): 2788 (60.6%) Spam (1): 1813 (39.4%)
Train / Test split	3450 / 1151 (75% / 25%, stratified)

The dataset was loaded programmatically via the official `ucimlrepo` package (with automatic fallback to a direct UCI archive download and a verified mirror of the same UCI data), so the notebook works end-to-end without depending on Kaggle.

4 Data Preprocessing

1. **Missing values:** Spambase has no missing values by construction; the pipeline defensively imputes with the column median if any are found.
2. **Normalization:** Features were standardized (zero mean, unit variance) for Gaussian NB and KNN; Min-Max scaled to $[0, 1]$ for Multinomial NB (which requires non-negative inputs); and binarized (threshold $= 0$) for Bernoulli NB.

5 Exploratory Data Analysis

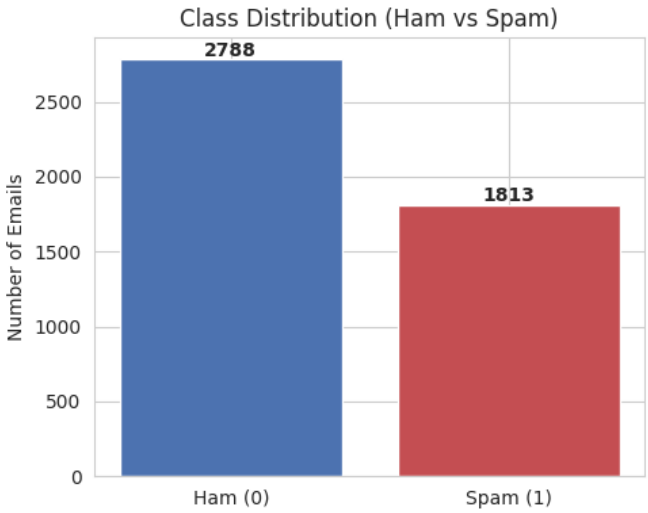


Figure 1: Class distribution: Ham (2788) vs. Spam (1813).

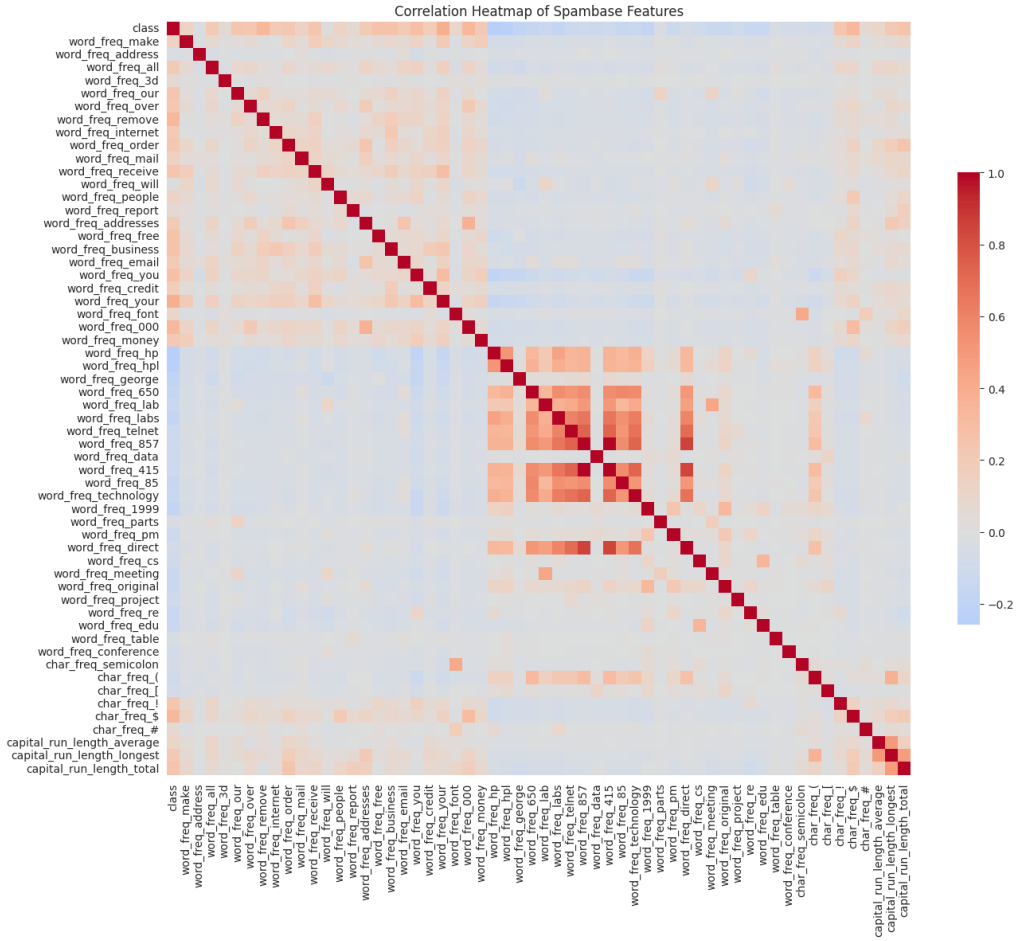


Figure 2: Correlation heatmap of the 57 Spambase features.

The features most correlated with the class label were `word_freq_your`, `word_freq_000`, `word_freq_remove`, `char_freq_$`, `word_freq_you`, and `word_freq_free`.

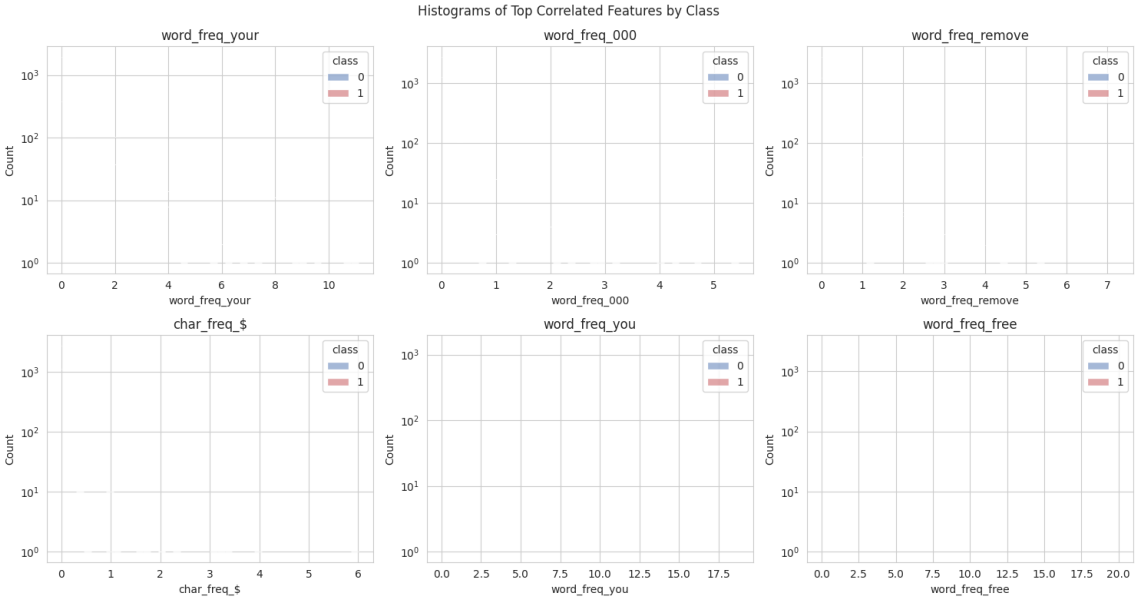


Figure 3: Histograms of the top correlated features, split by class.

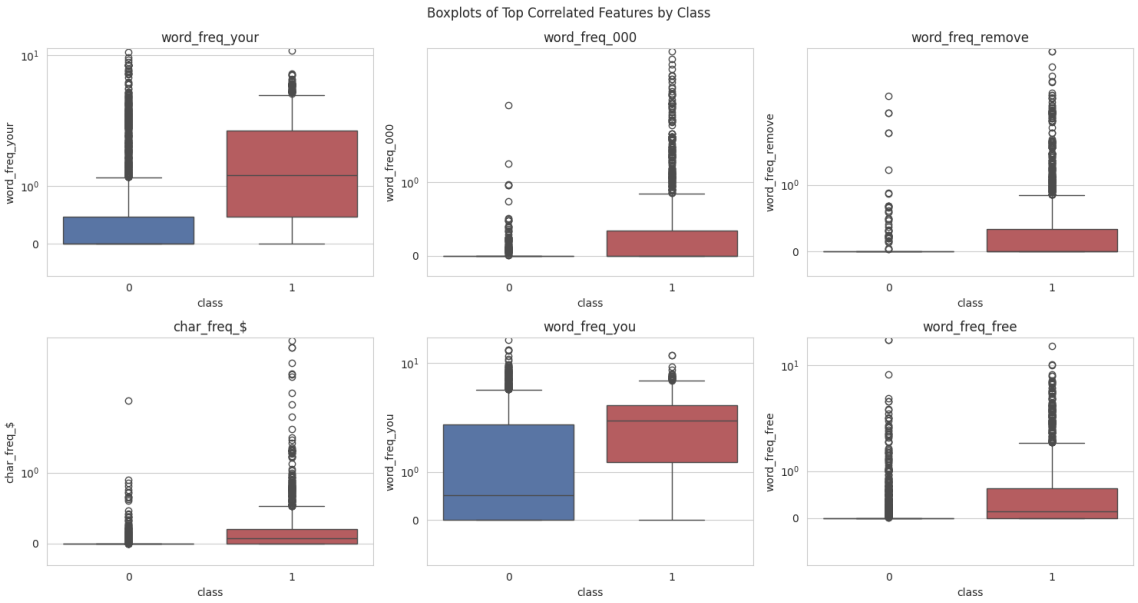


Figure 4: Boxplots of the top correlated features, split by class.

6 Implementation Overview

The following pipeline was implemented in Python (NumPy, Pandas, Matplotlib, Seaborn, scikit-learn):

1. Load the UCI Spambase dataset.
2. Handle missing values (none found; defensive median imputation included).
3. Normalize features (StandardScaler / MinMaxScaler / Binarizer as required per model).
4. Perform exploratory data analysis.

5. Stratified train/test split (75/25).
6. Train Gaussian NB, Multinomial NB, Bernoulli NB, and KNN.
7. Tune KNN using `GridSearchCV`.
8. Tune KNN using `RandomizedSearchCV`.
9. Compare KDTree and BallTree search algorithms.
10. Perform 5-fold stratified cross-validation.
11. Measure training and prediction time for every model.
12. Generate all required evaluation plots.

A representative code snippet for model evaluation:

```
def evaluate_model(name, model, X_tr, y_tr, X_te, y_te):
    t0 = time.perf_counter()
    model.fit(X_tr, y_tr)
    train_time = time.perf_counter() - t0

    t0 = time.perf_counter()
    y_pred = model.predict(X_te)
    predict_time = time.perf_counter() - t0

    y_proba = model.predict_proba(X_te)[:, 1]
    return {
        'Accuracy': accuracy_score(y_te, y_pred),
        'Precision': precision_score(y_te, y_pred),
        'Recall': recall_score(y_te, y_pred),
        'F1': f1_score(y_te, y_pred),
        'ROC-AUC': roc_auc_score(y_te, y_proba),
        'Train␣Time␣(s)': train_time,
        'Predict␣Time␣(s)': predict_time,
    }
```

The full, executed implementation is provided in the accompanying Jupyter notebook `Experiment_2_Spam_Class`

7 Results

7.1 Naïve Bayes Comparison

Table 2: Naïve Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8254	**0.9001**	0.8818
Precision	0.7077	**0.9472**	0.8768
Recall	**0.9493**	0.7907	0.8150
F1	0.8109	**0.8619**	0.8447
ROC-AUC	0.9368	**0.9614**	0.9503

Multinomial NB achieved the best accuracy, precision, F1 and ROC-AUC, matching the count-based nature of the frequency features. Gaussian NB obtained the highest recall but lowest precision, since its normal-distribution assumption is a poor fit for these sparse, skewed features.

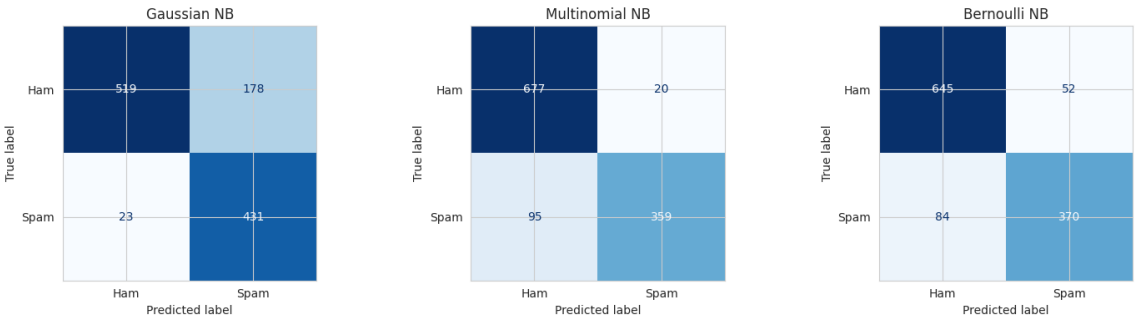


Figure 5: Confusion matrices for Gaussian, Multinomial, and Bernoulli NB.

7.2 KNN — Effect of Varying k

Table 3: KNN Comparison (Euclidean distance, uniform weights)

k	Accuracy	Precision	Recall	F1
1	0.8975	0.8733	0.8656	0.8695
3	0.9018	0.8814	0.8678	0.8746
5	0.8992	0.8789	0.8634	0.8711
7	0.9053	0.8929	0.8634	0.8779
9	**0.9062**	0.8932	0.8656	0.8792
11	0.9027	0.8904	0.8590	0.8744

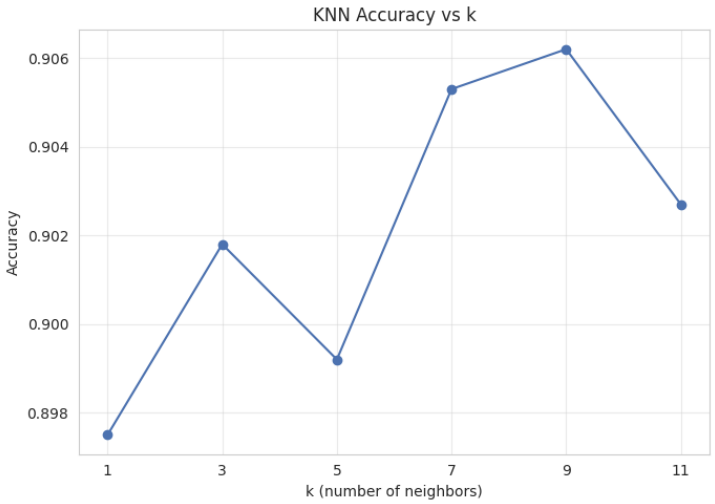


Figure 6: KNN accuracy vs. k. Best plain KNN: $k = 9$, accuracy $= 0.9062$.

7.3 Hyperparameter Tuning: GridSearchCV vs. RandomizedSearchCV

Table 4: GridSearchCV vs. RandomizedSearchCV (KNN)

Parameter	**GridSearchCV**	**RandomizedSearchCV**
Best k	13	6
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	ball_tree	kd_tree
CV Accuracy	0.9241	**0.9249**
Execution Time (s)	41.19	**16.82**

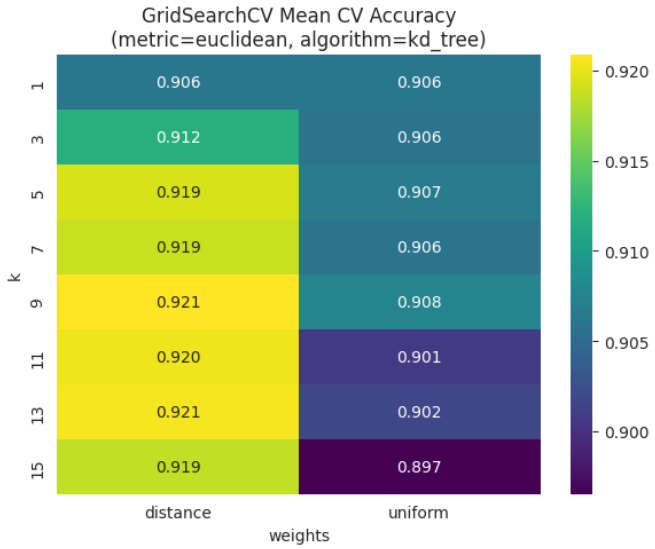


Figure 7: GridSearchCV mean CV accuracy heatmap (metric=euclidean, algorithm=kd_tree).

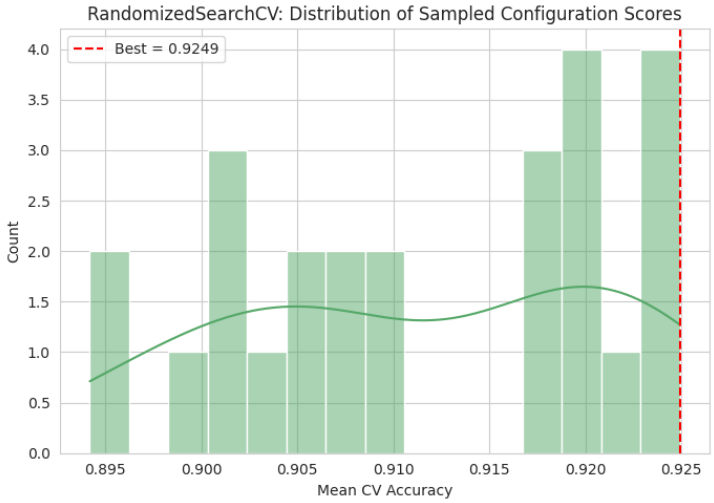


Figure 8: Distribution of mean CV accuracy across RandomizedSearchCV's sampled configurations.

Both searches converge on *Manhattan distance* with *distance-weighted* voting as the strongest configuration. RandomizedSearchCV reached a marginally higher CV accuracy in about 41% of GridSearchCV's execution time by sampling only 25 of the 64 possible grid combinations.

7.4 Best Tuned KNN

Best parameters (GridSearchCV): `algorithm=ball_tree, metric=manhattan, n_neighbors=13, weights=distance`.

Table 5: Best Tuned KNN — Full Evaluation

Metric	Value
Accuracy	0.9209
Precision	0.9504
Recall	0.8436
F1	0.8938
ROC-AUC	0.9754
Train Time (s)	0.0213
Predict Time (s)	0.2531

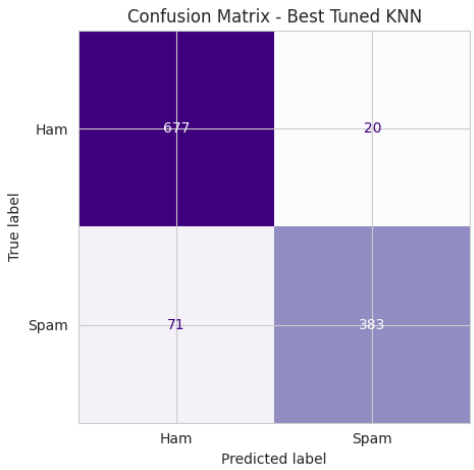


Figure 9: Confusion matrix — Best tuned KNN.

7.5 KDTree vs. BallTree

Table 6: KDTree vs. BallTree

Metric	KDTree	BallTree
Accuracy	0.9209	0.9209
Training Time (s)	0.0167	**0.0091**
Prediction Time (s)	0.2864	**0.2404**

Both structures give identical accuracy (they compute exact nearest neighbors); BallTree was faster for both training and prediction on this 57-dimensional dataset.

7.6 5-Fold Cross Validation

Table 7: 5-Fold Cross Validation: Best Naïve Bayes vs. Best KNN

Fold	Naïve Bayes (Multinomial)	Best KNN
1	0.8855	0.9130
2	0.8971	0.9232
3	0.8942	0.9420
4	0.8826	0.9261
5	0.8812	0.9159
Average	**0.8881**	**0.9240**

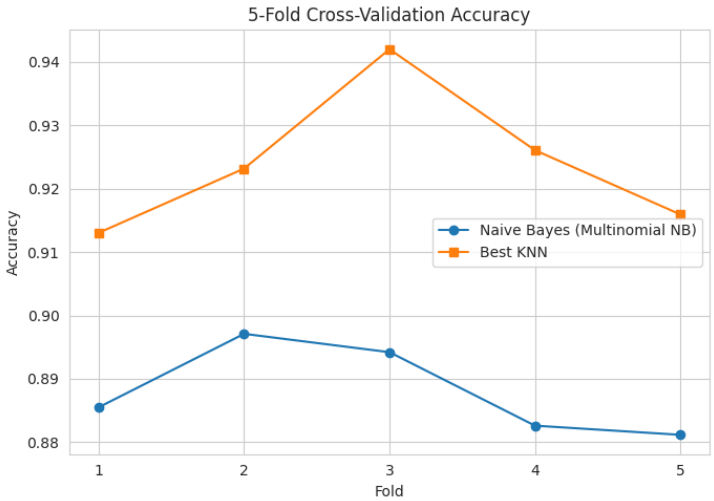


Figure 10: 5-Fold cross-validation accuracy per fold.

The tuned KNN model consistently outperforms Multinomial NB across every fold, confirming the stability of its higher test accuracy.

8 Complexity Analysis

8.1 Theoretical Time Complexity

Table 8: Theoretical Time Complexity (n = training samples, d = features)

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{\text{avg}}$
BallTree	$O(n \log n)$	$O(\log n)_{\text{avg}}$

8.2 Experimental Time Analysis

Table 9: Experimental Time Analysis ($n \approx 3450$ train, ≈ 1151 test, $d = 57$)

Algorithm	Training (s)	Prediction (s)
Gaussian NB	0.0144	0.0007
Multinomial NB	**0.0021**	**0.0002**
Bernoulli NB	0.0036	0.0011
Best KNN	0.0092	**0.2358**

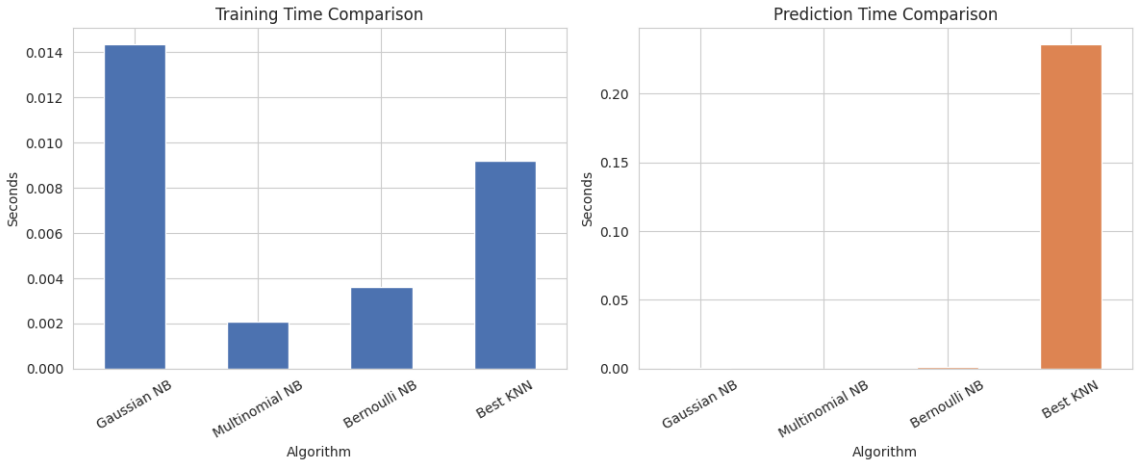


Figure 11: Training time and prediction time comparison across all models.

The measured times closely track theory: all Naïve Bayes variants train and predict in a few milliseconds, while KNN's prediction time is two to three orders of magnitude larger, since it must search neighbors at query time for every test point even with a tree-based index. KNN's training time is negligible, as it is a lazy learner that merely stores the training data (and, when applicable, builds the tree index).

9 Additional Evaluation Plots

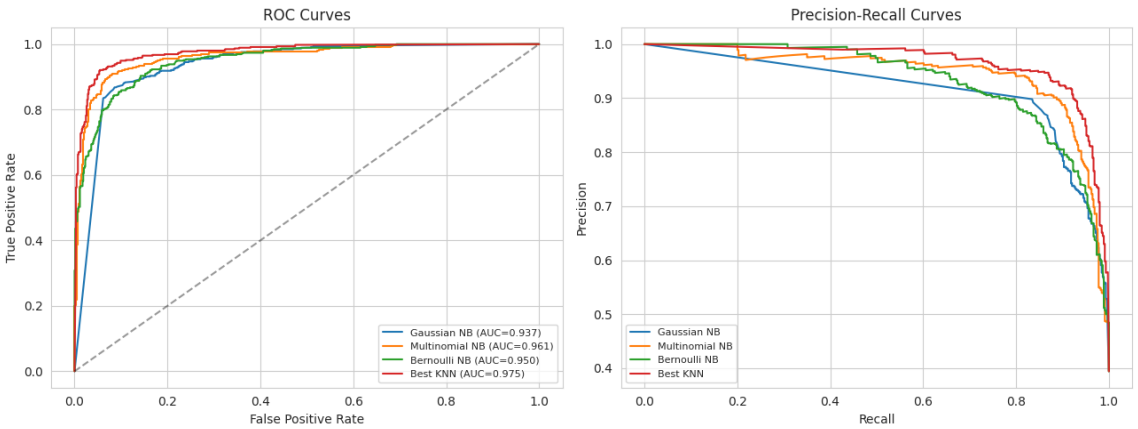


Figure 12: ROC curves and Precision-Recall curves for all models.

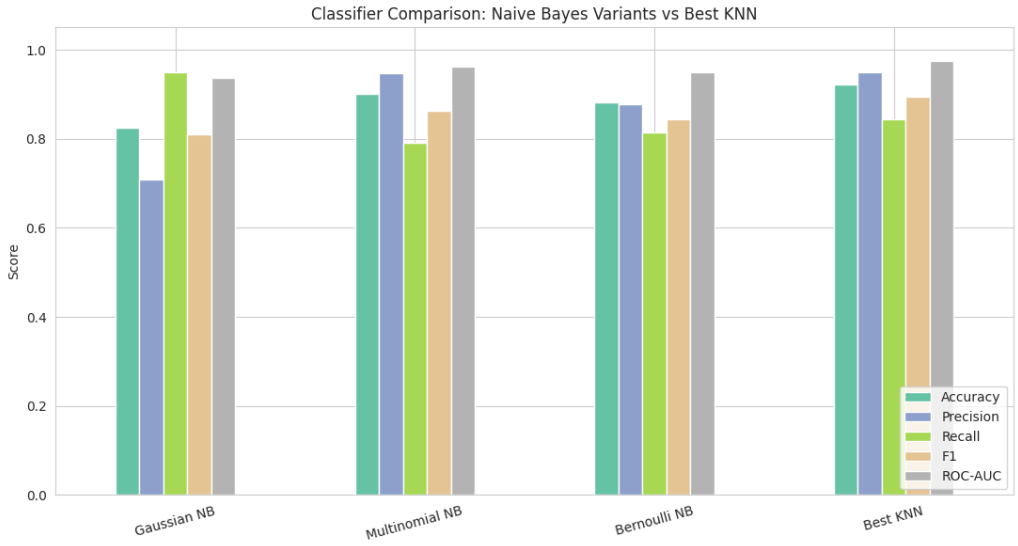


Figure 13: Overall classifier comparison across Accuracy, Precision, Recall, F1 and ROC-AUC.

Table 10: Overall Classifier Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gaussian NB	0.8254	0.7077	**0.9493**	0.8109	0.9368
Multinomial NB	0.9001	**0.9472**	0.7907	0.8619	0.9614
Bernoulli NB	0.8818	0.8768	0.8150	0.8447	0.9503
Best KNN (tuned)	**0.9209**	0.9504	0.8436	**0.8938**	**0.9754**

10 Additional Tasks

10.1 Effect of Train-Test Split Ratio (Tuned KNN)

Test Size	Train Size	Accuracy
0.1	0.9	0.9046
0.2	0.8	0.9164
0.3	0.7	**0.9247**
0.4	0.6	0.9158

Accuracy remains stable (0.90–0.92) across split ratios, peaking near a 70/30 split.

10.2 Euclidean vs. Manhattan Distance

Distance	Accuracy	F1
Euclidean	0.9140	0.8894
Manhattan	**0.9209**	**0.8938**

Manhattan distance outperforms Euclidean, consistent with the tuning result, likely because it is less sensitive to the outliers/skew present in the frequency features.

10.3 Weighted vs. Uniform KNN

Weighting	Accuracy	F1
Uniform	0.9027	0.8670
Distance	**0.9209**	**0.8938**

Distance-weighted voting clearly improves both accuracy and F1 over uniform voting.

10.4 Overfitting vs. Underfitting in KNN

A very small k (e.g. $k = 1$) makes KNN sensitive to noise and outliers in the training data, causing the decision boundary to become highly irregular and the model to **overfit**. A very large k oversmooths the boundary by averaging over many, possibly irrelevant, distant neighbors, causing the model to **underfit** and lose sensitivity to local structure. The accuracy-vs-k curve (Figure 5) reflects this: it rises then plateaus/dips, and the best generalizing configuration found by `GridSearchCV` used a moderate $k = 13$ combined with distance weighting.

11 Analysis Questions

1. **Which Naïve Bayes variant performed best?** Multinomial NB (Accuracy 0.9001, F1 0.8619, ROC-AUC 0.9614) — its count-based event model best matches word/character frequency features.

2. **What is the optimal value of k?** $k = 13$ with distance weighting and Manhattan distance (GridSearchCV); the best plain (unweighted, Euclidean) k was 9.

3. **Compare GridSearchCV and RandomizedSearchCV.** RandomizedSearchCV reached a slightly higher CV accuracy (0.9249 vs. 0.9241) in 16.82s vs. 41.19s ($\sim$2.4$\times$ faster) by sampling only 25 of 64 possible combinations.

4. **Compare KDTree and BallTree.** Identical accuracy (both exact); BallTree was faster here for both training and prediction.

5. **Compare theoretical and practical complexity.** Practical timings matched theory: Naïve Bayes trains/predicts in milliseconds; KNN prediction is by far the most expensive step, consistent with its $O(nd)$ / $O(\log n)_{\text{avg}}$ complexity versus Naïve Bayes' $O(d)$ prediction cost.

6. **Which classifier is preferred for large datasets?** Naïve Bayes — its linear-in-n training and near-constant prediction cost per query scale far better than KNN, whose prediction cost grows with the size of the reference dataset even with a tree index.

12 Conclusion

Across all metrics, the **tuned KNN model** ($k = 13$, Manhattan distance, distance-weighted, BallTree algorithm) was the strongest overall classifier (Accuracy 0.9209, F1 0.8938, ROC-AUC 0.9754), narrowly ahead of **Multinomial NB** (Accuracy 0.9001, ROC-AUC 0.9614). However, KNN's prediction time ($\sim$0.24 s for 1151 test points) is roughly 100–1000$\times$ slower than any Naïve Bayes variant, so for very large-scale or latency-sensitive spam filtering, **Multinomial Naïve Bayes** remains the more practical choice, while tuned KNN is preferable when maximum accuracy is the priority and the dataset size is moderate.

References

- Scikit-learn Documentation, https://scikit-learn.org
- Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*
- Christopher M. Bishop, *Pattern Recognition and Machine Learning*
- Dua, D. and Graff, C. (2019). UCI Machine Learning Repository, Spambase Data Set. Irvine, CA: University of California, School of Information and Computer Science. https://archive.ics.uci.edu/dataset/94/spambase